

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 2

Duration : 45 Minutes

Total Marks: 20

Annexure A

Descriptive Written Test

Code of Conduct:

I _S.Divyasri(192524388)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1

A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs — motion, temperature changes, and time of day — while also learning user preferences over time. Explain the four types of intelligent agents and analyse how each would function within this system, then justify which type, or hybrid combination, would be most effective for comfort, energy efficiency, and security.

1.1 Intelligent Agents in a Sensor-Driven Environment

An intelligent agent is any entity that perceives its environment through sensors and acts upon it through actuators in pursuit of a performance measure. In the system described above, the sensors are motion detectors, thermostats, light meters, and door or window contacts, while the actuators are lighting circuits, HVAC controllers, and locks. The four classical agent types differ mainly in how much of the environment they represent internally, and how that representation is used to select an action. Figure 1 illustrates the internal structure of each type as it would be wired into this particular home.

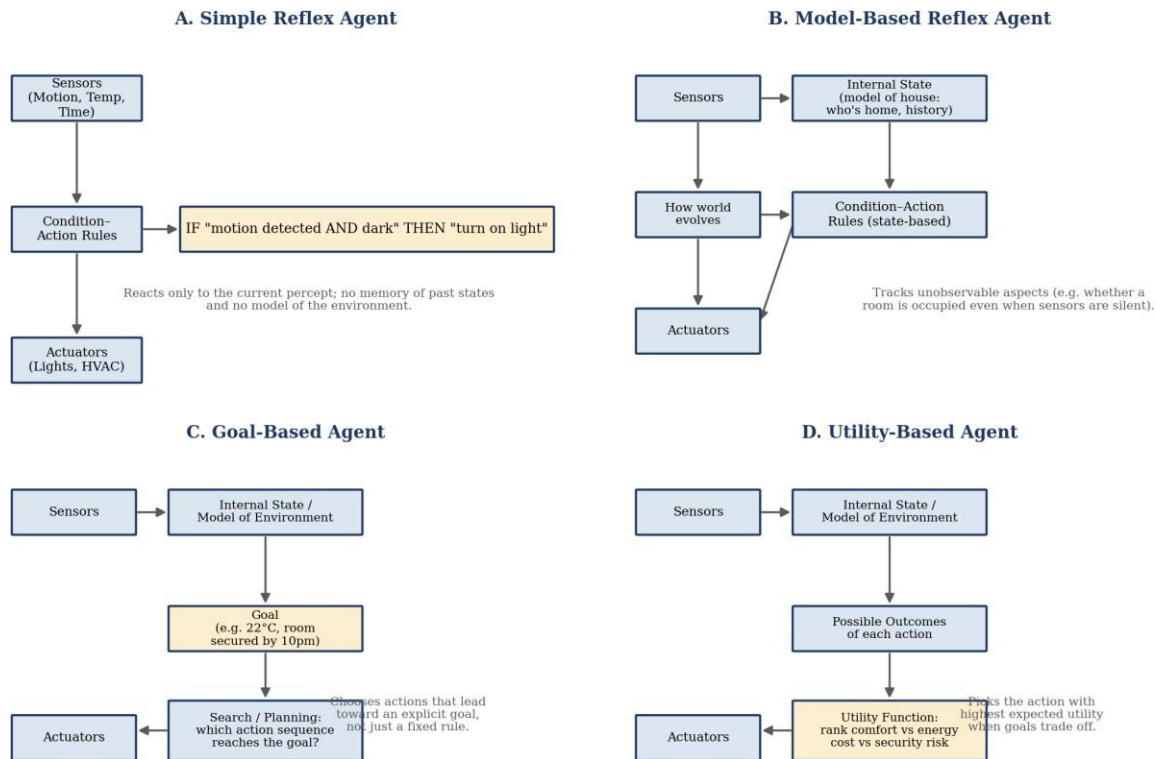


Figure 1. Internal architecture of the four agent types applied to the smart home scenario.

1.2 How Each Agent Type Would Function in This System

(a) Simple Reflex Agent

A simple reflex agent selects an action using only the current percept, matched against a fixed set of condition–action rules; it keeps no history and builds no model of the world. Within the smart home, it would implement direct rules such as switching on a light the instant a motion sensor fires in a dark room, or activating an alarm the moment a door contact opens after the security mode is armed. This makes the agent extremely fast and inexpensive to run, but it breaks down whenever the correct action depends on something the current percept does not reveal — for instance, a person sitting motionless while reading would be read as "room empty" and the light would switch off.

(b) Model-Based Reflex Agent

A model-based reflex agent extends the same rule-following behaviour with an internal state that tracks aspects of the environment which are not directly observable at every instant, such as which

rooms were recently occupied, the outdoor temperature trend, or whether a family member is expected home. This allows the system to keep a room lit for a reasonable interval after motion stops, or to anticipate a temperature drop overnight based on recent readings rather than reacting only once the room has already cooled. It handles partial observability far better than the simple reflex design, but it still lacks any explicit goal to work towards — it only ever reacts to state.

(c) Goal-Based Agent

A goal-based agent adds an explicit goal — for example, "reach 22°C by 6 p.m." or "ensure all entry points are secured by 10 p.m." — and searches over possible action sequences to find one that achieves it. This is what allows the system to plan ahead: rather than simply reacting to the current temperature, it can calculate when heating needs to start so the goal is met exactly on time, adjusting the plan if a window is found open partway through. The limitation is that a goal-based agent treats all goal-achieving plans as equally acceptable, even if one wastes far more energy than another.

(d) Utility-Based Agent

A utility-based agent goes one step further by attaching a numeric utility to each possible outcome, capturing trade-offs between competing objectives such as comfort, energy cost, and security risk. Instead of merely reaching 22°C by any means, it would compare several heating schedules and choose the one that reaches the target temperature while consuming the least energy and without compromising the security routine. This is essential wherever objectives conflict — for example, keeping a porch light on all night improves security but increases energy consumption, and only a utility-based evaluation can weigh that trade-off explicitly.

1.3 Justification: A Layered Hybrid Architecture

No single agent type is sufficient on its own. A purely reflex design cannot plan or optimise, while a purely utility-based design is too computationally heavy to run for split-second safety responses such as a smoke alarm. The most effective design is therefore a layered hybrid architecture, in which each layer handles the class of decision it is best suited to, as shown in Figure 2.

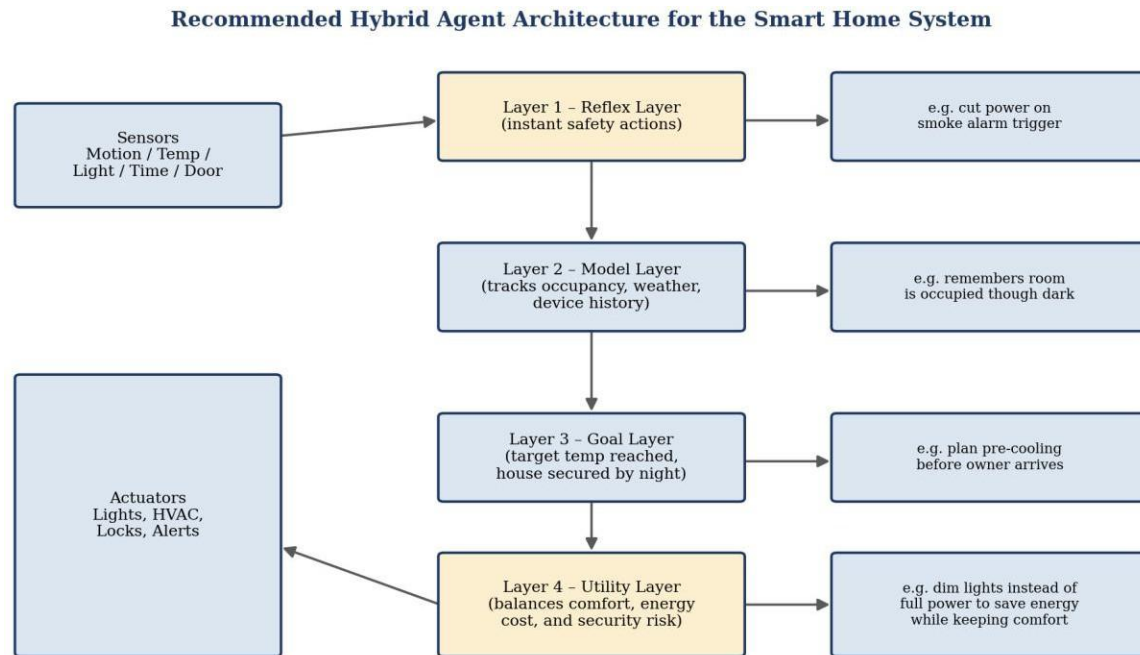


Figure 2. Proposed four-layer hybrid architecture combining reflex, model-based, goal-based, and utility-based reasoning.

At the base, a reflex layer guarantees an instant response to safety-critical events, such as cutting power when a smoke alarm triggers, without waiting for any planning process. Above it, a model layer maintains continuity — occupancy history, recent temperature trends, and device status — so decisions are informed rather than momentary. A goal layer then plans ahead for scheduled objectives such as pre-heating before the owner arrives, while a utility layer arbitrates whenever objectives compete, for example dimming rather than fully powering lights to save energy without sacrificing comfort. This layered combination gives the system the speed of a reflex agent, the continuity of a model-based agent, the foresight of a goal-based agent, and the balanced decision-making of a utility-based agent — which together best serve comfort, energy efficiency, and security simultaneously.

1.4 Comparative Summary

Agent Type	Perception & Decision Rule	Behaviour in the Smart Home	Strength	Limitation
Simple Reflex	Maps the current percept directly to an action through condition–action rules; retains no memory.	"If motion is detected and the room is dark, switch on the light."	Extremely fast, cheap, and predictable.	Fails when the environment is only partially observable (e.g., a still, silent occupant).
Model-Based Reflex	Maintains an internal model of unobserved aspects of the world, updated as new percepts arrive.	Remembers a room is occupied even once motion stops, so lights are not switched off prematurely.	Copes with partial observability.	Still purely reactive; no explicit objective to plan toward.
Goal-Based	Evaluates candidate action sequences against an explicit goal state and selects ones that achieve it.	Plans a heating sequence so the room reaches 22°C by the time the owner is expected home.	Flexible; supports planning and changing goals.	Treats every goal-achieving option as equally good; cannot rank them.
Utility-Based	Assigns a numeric utility to each outcome and chooses the action that maximises expected utility.	Weighs comfort against energy cost and security risk to select the best overall setting, not merely an acceptable one.	Handles trade-offs and uncertainty gracefully.	Harder to design; requires a carefully tuned utility function.

Question 2

A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies such as Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem, discuss their advantages and disadvantages in terms of time and space complexity, relate the problem to different agent types, and suggest the most effective combination of agent type and search strategy.

2.1 Framing the Maze as a Search Problem

Because the robot has no prior map, the maze must be treated as an uninformed search problem: the state space is the set of reachable cells, the successor function moves the robot to an adjacent open cell, and the goal test checks whether the current cell is the exit. Both UCS and IDS explore this space systematically, but they differ in which node they expand next and in how much memory that choice demands.

2.2 Uniform Cost Search (UCS)

UCS always expands the frontier node with the lowest cumulative path cost $g(n)$, using a priority queue ordered by cost rather than by depth. If some cells in the maze are more costly to cross than others — a longer corridor, a slower surface, or a cell with higher collision risk — UCS is the strategy that still guarantees the cheapest overall route, even if that route passes through more cells than a shorter but costlier alternative. Figure 3 traces this behaviour on a small maze graph: although the path through nodes A and D has fewer edges, UCS correctly returns the path through B and E because its total cost is lower.

Uniform Cost Search: Expanding the Maze Graph by Path Cost

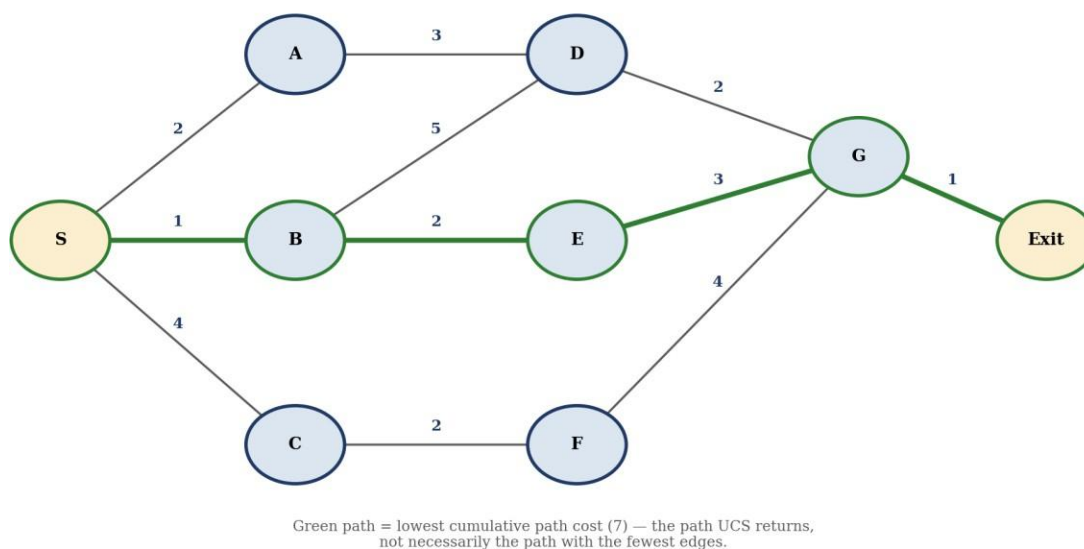


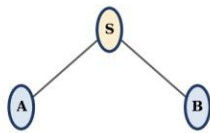
Figure 3. Uniform Cost Search selects the path with the lowest cumulative cost (highlighted in green), not necessarily the path with the fewest steps.

2.3 Iterative Deepening Search (IDS)

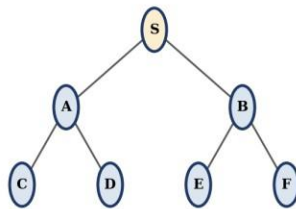
IDS combines the low memory usage of depth-first search with the completeness of breadth-first search by repeatedly running a depth-limited search, increasing the limit by one after every failed attempt, until the exit is found. At each iteration the robot re-explores the maze from the entrance, discarding everything learned in the previous pass, but it only ever needs to remember a single path from the entrance to its current position. Figure 4 shows three successive iterations over the same maze tree: the exit, located at depth 3, is only discovered once the depth limit reaches 3, after two shallower attempts have been exhausted.

Iterative Deepening Search: Re-exploring the Maze Tree at Increasing Depth Limits

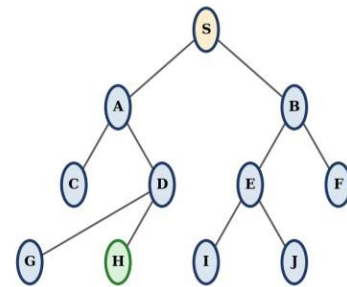
Iteration 1: Depth Limit = 1



Iteration 2: Depth Limit = 2



Iteration 3: Depth Limit = 3



Node H (the exit) is only discovered once the depth limit reaches 3; shallower iterations revisit S, A, B, C, D, E, F each time.

Figure 4. Iterative Deepening Search re-explores the maze at increasing depth limits until the exit is discovered.

2.4 Advantages and Disadvantages: Time and Space Complexity

The two strategies make opposite trade-offs. UCS is optimal whenever step costs are non-negative, but its time and space complexity is $O(b^{(1+\lceil C^*/\epsilon \rceil)})$, where C^* is the optimal path cost and ϵ the smallest step cost — this can be far worse than the actual solution depth if costs vary widely, and it must keep the entire frontier in memory. IDS runs in $O(b^d)$ time, similar to breadth-first search because shallow nodes are regenerated on every iteration, but its space complexity is only $O(bd)$, since it never needs more than one path in memory at a time. In a very large or memory-constrained maze, this makes IDS the safer choice; in a maze where cells carry meaningfully different costs, UCS is the only one of the two that is guaranteed optimal.

Criterion	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Strategy	Expands the frontier node with the lowest cumulative path cost $g(n)$, using a priority queue.	Repeats depth-limited depth-first search with an increasing limit (0, 1, 2, ...) until the goal appears.
Completeness	Complete whenever step costs are non-negative.	Complete for a finite maze with a finite branching factor.
Optimality	Optimal — always returns the least-cost path, valuable when corridors differ in length or hazard.	Optimal only when every step cost is equal, i.e., it finds the path with the fewest moves.
Time Complexity	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$; grows sharply if path costs vary widely across the maze.	$O(b^d)$; comparable to breadth-first search because shallow levels are regenerated every iteration.
Space Complexity	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$; the whole frontier must be held in a priority queue, which can be memory-heavy.	$O(bd)$; stores only the single path from the root to the current node — far more memory-efficient.
Best suited when	Passages carry unequal cost (e.g., slower or riskier cells) and memory is not tightly constrained.	Memory is limited and all moves cost the same, so depth alone reflects path length.

2.5 Relating the Problem to Agent Types

The robot in the maze is best modelled as a goal-based agent: it has a single, well-defined goal (reach the exit) and must search over action sequences to achieve it, which is exactly the role UCS and IDS play as the underlying planning mechanism. A simple reflex or model-based agent could only follow local rules such as "always turn left" — a strategy that may loop indefinitely in a maze with cycles and provides no guarantee of reaching the exit at all. A utility-based agent becomes relevant only if the maze introduces competing objectives beyond simply exiting, such as minimising battery consumption or avoiding damage-prone cells; in that case the utility function would fold path cost, time, and risk into a single measure that a cost-aware search such as UCS is naturally able to optimise. Agent design therefore directly shapes which search strategy is meaningful: a goal-based agent needs a complete search method, while a utility-based agent needs a cost-sensitive one.

2.6 Recommended Combination

For an unknown maze with variable passage costs and no strict memory limitation, a goal-based agent using Uniform Cost Search is the most effective combination, since it both guarantees reaching the exit and guarantees the cheapest route to it. If the robot instead operates under strict memory constraints — a realistic concern for small embedded hardware — and all moves can be treated as equal cost, a goal-based agent using Iterative Deepening Search becomes preferable, trading a higher but still bounded time cost for a dramatically smaller memory footprint. Where both memory is limited and costs are unequal, the strongest design upgrades the agent to a utility-based one and combines it with a cost-aware, depth-limited variant of these searches, so the robot balances the guarantee of reaching the exit against the practical constraints of the hardware it runs on.

Rubrics for Calculation

The following rubric was supplied with the assessment and governs how each answer above is evaluated across six weighted criteria.

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding with major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or missing explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or analysis shown
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization throughout
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect or largely irrelevant
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly presented and hard to follow